

```

  2 | 2023-02-01 | Linux |      1
(1 row)

forumdb=> select * from part_tags_date_03_2023;
pk | ins_date | tag | level
-----+-----+-----+-----
  3 | 2023-03-01 | BSD Unix |      1
(1 row)
forumdb=> select * from part_tags_date_04_2023;
pk | ins_date | tag | level
-----+-----+-----+-----
  4 | 2023-04-01 | Rocky Linux Distro |      2
(1 row)

```

As we can see, all the data has been partitioned correctly.

Partition maintenance

In the previous two sections, we saw what declarative partitioning is and how to create partitioned tables when we start our work from scratch. In this section, we'll examine how to attach or detach partitions when the partitioned table already exists. We will look at how to do the following:

- Attaching a new partition
- Detaching an existing partition
- Attaching an existing table to the parent table

Attaching a new partition

If we want to attach a new partition to the parent table, we have to execute the following:

```

forumdb=> CREATE TABLE part_tags_date_05_2023 PARTITION OF part_tags FOR
VALUES FROM ('2023-05-01') TO ('2023-05-30');
CREATE TABLE

```

As we can see here, a new partition called `part_tags_date_05_2023` has been added to the `part_tags` parent table:

```

forumdb=> \d+ part_tags;
        Partitioned table "forum.part_tags"

```

```

      Column |          Type          | [...] | Description
-----+-----+-----+-----
pk          | integer                | [...] |
ins_date    | date                   | [...] |
tag         | character varying(255) | [...] |
level       | integer                | [...] |

Partition key: RANGE (ins_date)
Indexes:
    "part_tags_pkey" PRIMARY KEY, btree (pk, ins_date)
    "part_tags_tag_idx" btree (tag)
Partitions: part_tags_date_01_2023 FOR VALUES FROM ('2023-01-01') TO
('2023-01-31'),
            part_tags_date_02_2023 FOR VALUES FROM ('2023-02-01') TO
('2023-02-28'),
            part_tags_date_03_2023 FOR VALUES FROM ('2023-03-01') TO
('2023-03-31'),
            part_tags_date_04_2023 FOR VALUES FROM ('2023-04-01') TO
('2023-04-30'),
            part_tags_date_05_2023 FOR VALUES FROM ('2023-05-01') TO
('2023-05-30')
```

Detaching an existing partition

If we want to detach an existing partition from the parent table, we have to execute the following:

```

forumdb=> ALTER TABLE part_tags DETACH PARTITION part_tags_date_05_2023 ;
ALTER TABLE
```

As we can see here, the partition called part_tags_date_05_2023 has been detached from the part_tags parent table:

```

forumdb=> \d+ part_tags;
                Partitioned table "forum.part_tags"
      Column |          Type          | [...] | Description
```

```

-----+-----+-----+-----
pk      | integer          | [...] |

ins_date | date              | [...] |

tag      | character varying(255) | [...] |

level    | integer           | [...] |

Partition key: RANGE (ins_date)
Indexes:
    "part_tags_pkey" PRIMARY KEY, btree (pk, ins_date)
    "part_tags_tag_idx" btree (tag)
Partitions: part_tags_date_01_2023 FOR VALUES FROM ('2023-01-01') TO
('2023-01-31'),
            part_tags_date_02_2023 FOR VALUES FROM ('2023-02-01') TO
('2023-02-28'),
            part_tags_date_03_2023 FOR VALUES FROM ('2023-03-01') TO
('2023-03-31'),
            part_tags_date_04_2023 FOR VALUES FROM ('2023-04-01') TO
('2023-04-30')

```

Attaching an existing table to the parent table

To practice this we need a table called `part_tags_already_exists` present in our database and containing all the tags with an entry date prior to 2022-12-31. If you are using the Docker image, you can find this inside the `forumdb` database. Otherwise, make sure to create the table with the following structure:

```

forumdb=> \d part_tags_already_exists
          Table "forum.part_tags_already_exists"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk       | integer                |           |          |
ins_date | date                   |           |          | not null

```

```
tag      | character varying(255) |      | not null  
  
level    | integer                 |      | not null
```

Indexes:

```
"part_tags_already_exists_pkey" PRIMARY KEY, btree (pk, ins_date)  
"part_tags_already_exists_tag_idx" btree (tag)
```

If we want to attach this table containing all the tags with a date entered prior to 2022-12-31 to the parent table, we have to run this statement:

```
forumdb=> ALTER TABLE part_tags ATTACH PARTITION part_tags_already_exists  
FOR VALUES FROM ('1970-01-01') TO ('2022-12-31');  
ALTER TABLE
```

In this way, the `part_tags_already_exists` table becomes a child table for the parent table, `part_tags`.

The default partition

In this section, we will see what happens if we insert data into a partitioned table where the child partition does not exist, and how to resolve the inconvenience this causes. To simulate this problem, suppose we want to insert a date corresponding to 2023-05-01 on the table called `part_tags`. We would get this result:

```
forumdb=> insert into part_tags (tag,ins_date,level) values ('Ubuntu  
Linux','2023-05-01',2);  
ERROR:  no partition of relation "part_tags" found for row  
DETAIL:  Partition key of the failing row contains (ins_date) = (2023-05-  
01).
```

This happens because PostgreSQL does not have a correspondence between the date of 2023-05-01 and those present on the mapping of the child tables.

To eliminate this drawback, it is necessary to use a default partition where all the values that are not reflected in the mapping of the child tables will be inserted.

To do this let's execute the following statement:

```
forumbd=> CREATE TABLE part_tags_default PARTITION OF part_tags default;
CREATE TABLE
```

Now let's try to repeat the previous entry:

```
forumbd=> insert into part_tags (tag,ins_date,level) values ('Ubuntu
Linux','2023-05-01',2);
INSERT 0 1
```

At this point, the data has been inserted in the default partition and is visible from the `part_tags` parent table, as we can see here:

```
forumbd=> select * from part_tags;
pk | ins_date | tag | level
----+-----+-----+-----
1 | 2023-01-01 | Operating Systems | 0
2 | 2023-02-01 | Linux | 1
3 | 2023-03-01 | BSD Unix | 1
4 | 2023-04-01 | Rocky Linux Distro | 2
6 | 2023-05-01 | Ubuntu Linux | 2
(5 rows)

forumbd=> select * from part_tags_default ;
pk | ins_date | tag | level
----+-----+-----+-----
6 | 2023-05-01 | Ubuntu Linux | 2
(1 row)
```

Partitioning and tablespaces

Now suppose we want to use the tablespaces seen in *Chapter 2* together with the partitioning procedure we have just seen. Using this technique, we will be able to place child tables on different tablespaces and, therefore, on different directories that could be mounted on different volumes.

This way of working can increase read/write performance. In the following example, we will limit ourselves to creating two tablespaces on local directories. However, it is not difficult, using the `mount` command, to map these two directories on different volumes. If you are using the Docker images provided with this chapter, the two directories we will use are already available.

If you aren't using the Docker images, you will first need to create two directories, `/data/tablespaces/ts_b` and `/data/tablespaces/ts_b`, where the `postgres` system user is able to read and write data.

Now let's connect to the `forumdb` database as the `postgres` user and create two tablespaces called `ts_a` and `ts_b`:

```
postgres@learn_postgresql:~$ psql -U postgres forumdb

forumdb=# create tablespace ts_a location '/data/tablespaces/ts_a';
CREATE TABLESPACE
forumdb=# create tablespace ts_b location '/data/tablespaces/ts_b';
CREATE TABLESPACE
```

Let's assign ownership to the `postgres` user:

```
forumdb=# alter tablespace ts_a owner to forum ;
ALTER TABLESPACE
forumdb=# alter tablespace ts_b owner to forum ;
ALTER TABLESPACE
```

Now let's reconnect to the `forumdb` database as the `forum` user:

```
forumdb=# \q
postgres@learn_postgresql:~$ psql -U forum forumdb
```

As in the previous case, let's recreate the parent table:

```
forumdb=> CREATE TABLE tablespace_part_tags (
    pk serial NOT NULL,
    ins_date date not null default now()::date,
    tag VARCHAR (255) NOT NULL,
    level INTEGER NOT NULL DEFAULT 0,
```

```

        primary key (pk,ins_date)
    )
    PARTITION BY RANGE (ins_date);
CREATE TABLE

```

Now let's create two child tables and one default table. The first child table will be created on tablespace ts_a and the second on tablespace ts_b:

```

forumbd=> CREATE TABLE tablespace_part_tags_date_2022 PARTITION OF
tablespace_part_tags FOR VALUES FROM ('2021-01-01') TO ('2022-12-31')
TABLESPACE ts_a;
CREATE TABLE
forumbd=> CREATE TABLE tablespace_part_tags_date_2023 PARTITION OF
tablespace_part_tags FOR VALUES FROM ('2023-01-01') TO ('2023-12-31')
TABLESPACE ts_b;
CREATE TABLE
forumbd=> CREATE TABLE tablespace_part_tags_date_default PARTITION OF
tablespace_part_tags default;
CREATE TABLE

```

Now, let's insert some data:

```

forumbd=> insert into tablespace_part_tags (tag,ins_date,level) values
('Operating Systems','2022-01-01',0), ('Linux','2022-02-01',1),('BSD
Unix','2023-03-01',1),('Rocky Linux Distro','2018-04-01',2);
INSERT 0 4

```

Then, let's see where the records have been stored:

```

forumbd=> select * from tablespace_part_tags;
pk | ins_date | tag | level
----+-----+-----+-----
1 | 2022-01-01 | Operating Systems | 0
2 | 2022-02-01 | Linux | 1
3 | 2023-03-01 | BSD Unix | 1
4 | 2018-04-01 | Rocky Linux Distro | 2
(4 rows)

```

```

forumbd=> select * from tablespace_part_tags_date_2022 ;
pk | ins_date | tag | level

```

```

-----+-----+-----+-----
  1 | 2022-01-01 | Operating Systems |      0
  2 | 2022-02-01 | Linux              |      1
(2 rows)

forumdb=> select * from tablespace_part_tags_date_2023 ;
pk | ins_date | tag      | level
-----+-----+-----+-----
  3 | 2023-03-01 | BSD Unix |      1
(1 row)

forumdb=>select * from tablespace_part_tags_date_default;
pk | ins_date | tag      | level
-----+-----+-----+-----
  4 | 2018-04-01 | Rocky Linux Distro |      2
(1 row)

```

As we have seen in this exercise, the data has been split into different tablespaces, and as a result we have doubled the speed. This is a very effective technique.

A simple case study

In this last section, we will not use the `forumdb` database. The database we will use instead is called `world_temperatures`, for which the public data has been imported from the public CSV present at <https://www.meteoblue.com/it/tempo/archive/export>.

The `db-world-temperatures` database backup can be found on the `packtpub` GitHub in the `chapter 9` directory, in the file called `backup-db-world-temperatures.sql.gz`. If you're using the Docker image, you will already have everything available; otherwise, to import the database, run PostgreSQL on your server:

```
$ gunzip < backup-db-world-temperatures.sql.gz | psql
```

If you are using the Docker image, just execute the following:

```
postgres@learn_postgresql:~$ psql -U postgres world_temperatures
```

Now, you will have the `db-world-temperatures` database ready to use. Inside the database, you will find an unpartitioned table named `basilea` and a partitioned table named `basilea_partitioned`; both tables contain temperature information for the city of Basel from 1950 to 2022 sampled at regular hourly intervals. Now, let's see the differences in behavior between searching a partitioned table and a non-partitioned table. Before continuing with the exercise, if you are not already familiar with the behavior of the `EXPLAIN` statement, check out *Chapter 13, The EXPLAIN Statement*.

Let's start by using the non-partitioned table and write a query that returns as a result the average temperature of the 5 coldest years for the period starting from 1950:

```
world_temperatures=# select extract (year from insert_time) as year,
avg(temperature) avg_temp from basilea group by 1 order by 2 limit 5;
 year |      avg_temp
-----+-----
1956 | 8.8073832344034608
1963 | 8.90779777708904110
1980 | 9.3459840948315118
1969 | 9.3705488990867580
1972 | 9.3749401615437158
(5 rows)
```

Likewise, let's get the 5 warmest years:

```
world_temperatures=# select extract (year from insert_time) as year,
avg(temperature) avg_temp from basilea group by 1 order by 2 desc limit
5;
 year |      avg_temp
-----+-----
2022 | 12.7320820592465753
2018 | 12.5638742964611872
2020 | 12.3662106902322404
2014 | 12.0601722329908676
2015 | 11.9246379973744292
(5 rows)
```

Let's take the last one as an example and see how it's done internally:

```
world_temperatures=# explain analyze select extract (year from insert_
time) as year, avg(temperature) avg_temp from basilea group by 1 order by
2 desc limit 5;
```

QUERY PLAN

```
-----
Limit (cost=98293.21..98293.23 rows=5 width=64) (actual
time=380.284..380.286 rows=5 loops=1)
  -> Sort (cost=98293.21..99892.99 rows=639912 width=64) (actual
time=380.282..380.283 rows=5 loops=1)
    Sort Key: (avg(temperature)) DESC
    Sort Method: top-N heapsort  Memory: 25kB
    -> HashAggregate (cost=68067.20..87664.51 rows=639912 width=64)
(actual time=380.121..380.253 rows=73 loops=1)
      Group Key: EXTRACT(year FROM insert_time)
      Planned Partitions: 32  Batches: 1  Memory Usage: 817kB
      -> Seq Scan on basilea (cost=0.00..12074.90 rows=639912
width=40) (actual time=0.030..189.680 rows=639912 loops=1)
Planning Time: 0.170 ms
Execution Time: 380.480 ms
(10 rows)
```

As we can see, PostgreSQL performs a sequential scan on the whole table. Now consider the partitioned table, `basilea_partitioned`:

```
world_temperatures=# \d+ basilea_partitioned
```

Partitioned

```
table "public.basilea_partitioned"
  Column      |          Type          | [...]
-----+-----+-----+
 id           | integer                | [...]
 insert_time  | timestamp with time zone | [...]
```